

Colossus Ansible Runbook — Phases 5B-1 & 5B-2

Document Type: Operational Runbook

Phase: 5B-1 (Foundation) + 5B-2 (Infrastructure Roles)

Author: Colossus Infrastructure Team

Date: 2026-02-14

Status: Phase 5B-1  COMPLETE | Phase 5B-2  COMPLETE

Depends on: Phase 5A (Traefik) , Phase 5A-TrueNAS 

1. Purpose

This runbook documents every configuration change made during Phases 5B-1 and 5B-2 to establish Ansible as the configuration management and infrastructure-as-code foundation for the Colossus homelab. It captures the exact commands, workarounds, troubleshooting steps, and lessons learned so the entire process can be reproduced or audited without repeating the trial-and-error that occurred during the initial implementation.

Phase 5B-1 scope (Foundation):

- SSH key authentication deployed to all 11 managed hosts
- SSH server installed and enabled on 3 LXC containers
- Python3 + libselinux-python3 layered onto 2 CoreOS VMs
- SSH connection multiplexing configured on workstation
- Ansible project structure created with inventory, vault, and validation playbook
- UniFi UDM SE IDS/IPS detection exclusions added for internal subnets
- Ansible configuration tuned (forks, vault auto-decrypt, output format)

Phase 5B-2 scope (Infrastructure Roles):

- `traefik-route` role — manages Traefik dynamic routing configuration via template
 - `pihole-dns` role — manages Pi-hole DNS records via REST API
 - `pbs-backup` role — manages Proxmox backup job definitions via template
 - `proxmox-vm` role — provisions CoreOS VMs on Proxmox hosts via `qm` commands
 - `proxmox-lxc` role — provisions LXC containers on Proxmox hosts via `pct` commands
 - `vars/` directory — parameterized rebuild recipes for all 4 VMs and 3 LXCs
 - Existing `deploy-app` + `colossus-legal` roles confirmed as the application deployment pattern
-

2. Prerequisites

2.1 Control Node (proxima-centauri)

The Ansible control node is `proxima-centauri`, Roman's Linux workstation on the main network (10.10.0.0/24).

Component	Version	Install Method
Ansible	2.16.3	System package (pre-existing)
Python3	3.x	System (pre-existing)
proxmoxer	2.2.0	<code>pip3 install proxmoxer --break-system-packages</code>
community.general	Pre-installed	<code>ansible-galaxy collection install community.general</code>

2.2 SSH Key

The workstation uses an Ed25519 SSH key:

```
~/.ssh/id_ed25519 (private)
~/.ssh/id_ed25519.pub (public)
SHA256:FEe4MIYzbkM5i8viYDAjzHfZT6kFGOfPqMaHqxrWav4
```

If regenerating on a new workstation:

```
bash
ssh-keygen -t ed25519 -C "roman@proxima-centauri"
```

3. SSH Key Deployment — All Hosts

3.1 Proxmox Nodes (root@)

Hosts: pve-1 (10.10.100.3), pve-2 (10.10.100.2), pve-3 (10.10.100.5)

pve-1 already had key auth configured. For pve-2 and pve-3, the standard `ssh-copy-id` approach had issues (stale host keys from pve-3 NIC replacement, temp directory errors). The working procedure was:

```
bash
```

```
# Clear stale host keys if needed (e.g., after NIC swap)
ssh-keygen -R 10.10.100.5
ssh-keygen -R 10.10.100.2

# If ssh-copy-id works:
ssh-copy-id root@10.10.100.2
ssh-copy-id root@10.10.100.5

# If ssh-copy-id fails (temp directory errors), push key manually:
cat ~/.ssh/id_ed25519.pub | ssh root@10.10.100.2 "cat >> /root/.ssh/authorized_keys"
ssh root@10.10.100.2 "chmod 600 /root/.ssh/authorized_keys"

# Verify key auth (should NOT prompt for password):
ssh -o ConnectTimeout=3 root@10.10.100.3 "hostname" # pve-1
ssh -o ConnectTimeout=3 root@10.10.100.2 "hostname" # pve-2
ssh -o ConnectTimeout=3 root@10.10.100.5 "hostname" # pve-3
```

Gotcha: `ssh-copy-id` can report "All keys were skipped because they already exist" even when the `authorized_keys` file doesn't exist. This is a misleading error caused by temp directory creation failures. Always verify by actually testing key-based SSH.

3.2 PBS (root@10.10.100.242)

PBS had no `authorized_keys` file despite `ssh-copy-id` claiming success:

```
bash

# Verify the file actually exists:
ssh root@10.10.100.242 "ls -la /root/.ssh/authorized_keys"

# If missing, create it manually:
cat ~/.ssh/id_ed25519.pub | ssh root@10.10.100.242 "cat >> /root/.ssh/authorized_keys && chmod 600 /root/.ssh/authorized_

# Verify:
ssh -o ConnectTimeout=3 root@10.10.100.242 "hostname"
```

3.3 CoreOS VMs (core@)

Hosts: VM-110 (10.10.100.110), VM-120 (10.10.100.120), VM-210 (10.10.100.200), VM-220 (10.10.100.220)

CoreOS VMs were provisioned with SSH keys via Butane/Ignition during initial creation, so key auth already worked for the `core` user:

```
bash
```

```
ssh -o ConnectTimeout=3 core@10.10.100.110 "hostname" # colossus-prod-db1
ssh -o ConnectTimeout=3 core@10.10.100.120 "hostname" # colossus-prod-app1
ssh -o ConnectTimeout=3 core@10.10.100.200 "hostname" # colossus-dev-db1
ssh -o ConnectTimeout=3 core@10.10.100.220 "hostname" # colossus-dev-app1
```

Gotcha: VM-220 (colossus-dev-app1) initially showed intermittent SSH timeouts from the workstation. This resolved after fixing the UniFi IPS issue (Section 8). If a CoreOS VM is reachable from its Proxmox host but not from the workstation, suspect IPS/IDS first.

3.4 LXC Containers (root@) — Required SSH Setup

Hosts: CT-311 pihole (10.10.100.53), CT-312 cloudflared (10.10.100.54), CT-313 traefik (10.10.100.55)

LXC containers did **not** have SSH configured at creation time. Ansible requires direct SSH access (not `pct exec`), so SSH had to be installed and keys deployed via `pct exec` through pve-3.

Step 1: Create .ssh directories

```
bash

ssh root@10.10.100.5 "pct exec 311 -- bash -c 'mkdir -p /root/.ssh && chmod 700 /root/.ssh'"
ssh root@10.10.100.5 "pct exec 312 -- bash -c 'mkdir -p /root/.ssh && chmod 700 /root/.ssh'"
ssh root@10.10.100.5 "pct exec 313 -- bash -c 'mkdir -p /root/.ssh && chmod 700 /root/.ssh'"
```

Step 2: Deploy SSH public key

```
bash

PUBKEY=$(cat ~/.ssh/id_ed25519.pub)

ssh root@10.10.100.5 "pct exec 311 -- bash -c 'echo \"$PUBKEY\" > /root/.ssh/authorized_keys && chmod 600 /root/.ssh/authorized_keys'"
ssh root@10.10.100.5 "pct exec 312 -- bash -c 'echo \"$PUBKEY\" > /root/.ssh/authorized_keys && chmod 600 /root/.ssh/authorized_keys'"
ssh root@10.10.100.5 "pct exec 313 -- bash -c 'echo \"$PUBKEY\" > /root/.ssh/authorized_keys && chmod 600 /root/.ssh/authorized_keys'"
```

Gotcha: Piping `cat ~/.ssh/id_ed25519.pub` through multiple SSH hops can append stray characters (tilde `~`). Using a variable (`PUBKEY=...`) with `echo` is cleaner and avoids this.

Step 3: Install and enable SSH server

The LXC containers (Debian 12) had `openssh-server` already installed but the SSH service was not enabled/running.

```
bash
```

```
ssh root@10.10.100.5 "pct exec 311 -- systemctl enable --now ssh"
ssh root@10.10.100.5 "pct exec 312 -- systemctl enable --now ssh"
ssh root@10.10.100.5 "pct exec 313 -- systemctl enable --now ssh"
```

Gotcha — Debian vs RHEL service name: Debian uses `ssh` (not `sshd`). Attempting `systemctl enable --now sshd` will fail with: `Refusing to operate on alias name or linked unit file: sshd.service`. The perl locale warnings during enable are cosmetic and can be ignored.

Step 4: Verify direct SSH access from workstation

```
bash

ssh -o ConnectTimeout=3 -o StrictHostKeyChecking=accept-new root@10.10.100.53 "hostname" # pihole
ssh -o ConnectTimeout=3 -o StrictHostKeyChecking=accept-new root@10.10.100.54 "hostname" # cloudflared
ssh -o ConnectTimeout=3 -o StrictHostKeyChecking=accept-new root@10.10.100.55 "hostname" # traefik
```

Use `-o StrictHostKeyChecking=accept-new` on first connection to auto-accept the new host keys.

4. Python3 on CoreOS VMs (Ansible Requirement)

4.1 The Problem

Fedora CoreOS is intentionally minimal — no Python installed by default. Ansible requires Python on managed hosts for all modules except `raw`. The App VMs (VM-120, VM-220) already had Python3 from their Butane provisioning, but the DB VMs (VM-110, VM-210) did not.

4.2 Canonical Solution: rpm-ostree with --apply-live

The community-consensus approach for Fedora CoreOS + Ansible is to layer `python3` and `libselinux-python3` using `rpm-ostree`. The `--apply-live` flag applies the change immediately via an overlayfs on `/usr`, avoiding a reboot.

```
bash

ssh core@10.10.100.110 "sudo rpm-ostree install python3 libselinux-python3 --apply-live"
ssh core@10.10.100.200 "sudo rpm-ostree install python3 libselinux-python3 --apply-live"
```

Why both packages:

- `python3` — Required for Ansible module execution
- `libselinux-python3` — Required for Ansible to manage files on SELinux-enforcing systems (all CoreOS VMs have SELinux enforcing)

4.3 How --apply-live Works

- Creates a new ostree deployment with the package layered on
- Mounts an overlayfs over `(/usr)` to make the packages available immediately
- The overlayfs is transient (rebuilt on reboot), but the package is persisted in the ostree deployment
- After next reboot, the package is available natively (no overlayfs needed)
- Survives CoreOS auto-updates — the package layer is maintained across version upgrades

4.4 Verification

```
bash

ssh core@10.10.100.110 "python3 --version"
ssh core@10.10.100.200 "python3 --version"

# Verify via Ansible:
ansible db_vms -m ping
```

4.5 Future VMs — Bake into Butane/Ignition

For new CoreOS VMs, the best practice is to add a one-shot systemd unit to the Butane config that installs Python before SSH becomes available. This ensures Ansible can manage the VM from first boot:

```
yaml

# Add to Butane file — runs once before sshd starts
systemd:
  units:
    - name: install-python-for-ansible.service
      enabled: true
      contents: |
        [Unit]
        Requires=network-online.target
        After=network-online.target
        Before=sshd.service
        [Service]
        Type=oneshot
        ExecCondition=/usr/bin/test ! -f /etc/python3-for-ansible.done
        ExecStart=/usr/bin/rpm-ostree install python3 libselenium-python3 --apply-live
        ExecStartPost=/usr/bin/touch /etc/python3-for-ansible.done
        [Install]
        WantedBy=multi-user.target
```

5. SSH Connection Multiplexing

5.1 The Problem

Ansible spawns multiple parallel SSH connections. Without connection multiplexing, each connection performs a full TCP + SSH handshake. Combined with IPS inspection (Section 8), this caused random SSH timeouts.

5.2 Configuration

Added to `~/.ssh/config` on proxima-centauri:

```
Host 10.10.100.*
  ControlMaster auto
  ControlPath ~/.ssh/cm-%r@%h:%p
  ControlPersist 120s
  ServerAliveInterval 15
  ServerAliveCountMax 3
```

What this does:

- `ControlMaster auto` — First connection to a host opens a persistent control socket; subsequent connections reuse it
- `ControlPath` — Socket file location (unique per user@host:port)
- `ControlPersist 120s` — Keep the master connection alive for 2 minutes after the last session disconnects
- `ServerAliveInterval 15` — Send keepalive every 15 seconds to detect dead connections
- `ServerAliveCountMax 3` — Close after 3 missed keepalives (45 seconds)

5.3 Pre-existing Entries

The config already had specific entries for 10.10.100.110 (VM-110) and 10.10.100.3 (pve-1) from earlier troubleshooting. The wildcard `10.10.100.*` entry covers all hosts. SSH matches the first applicable `Host` entry, so specific entries take precedence over the wildcard.

5.4 Managing Stale Sockets

If connections hang due to a stale multiplexing socket:

```
bash
```

```
# List active sockets:
```

```
ls ~/.ssh/cm-*
```

```
# Kill a specific socket:
```

```
ssh -O exit -o ControlPath=~/.ssh/cm-%r@%h:%p root@10.10.100.5
```

```
# Nuclear option — kill all:
```

```
rm ~/.ssh/cm-*
```

6. Ansible Project Structure

6.1 Directory Layout

```
~/colossus-ansible/
├── ansible.cfg          # Ansible configuration
├── inventory/
│   ├── hosts.yml        # All hosts, groups, variables
│   └── group_vars/
│       ├── all.yml      # Global variables (domain, IPs, deploy settings)
│       ├── coreos_vms.yml # ansible_user: core, python interpreter
│       ├── infrastructure.yml # ansible_user: root
│       ├── proxmox.yml   # PBS backup job definitions (cluster-wide)
│       ├── dev.yml       # DEV environment overrides
│       ├── prod.yml      # PROD environment overrides
│       └── vault.yml     # Encrypted variables
├── host_vars/
│   ├── traefik.yml       # Traefik route definitions (5 routes)
│   └── pihole.yml        # Pi-hole DNS record definitions (11 records)
├── secrets/
│   └── vault.yml         # Ansible Vault encrypted secrets
├── playbooks/
│   ├── gather-facts.yml  # Validation playbook (all hosts)
│   ├── deploy-app.yml    # Application deployment orchestrator
│   ├── manage-traefik.yml # Traefik route configuration
│   ├── manage-pihole.yml # Pi-hole DNS record management
│   ├── manage-pbs-backups.yml # PBS backup job configuration
│   ├── create-vm.yml     # CoreOS VM provisioning
│   └── create-lxc.yml    # LXC container provisioning
├── roles/
│   ├── deploy-app/      # Generic deployment orchestrator
│   └── tasks/{deploy,validate,rollback}.yaml
```



```

| | └─ templates/deployment-manifest.json.j2
| └─ colossus-legal/          # App-specific role (Quadlet, env, config)
| └─ traefik-route/          # Traefik dynamic routing (template-based)
| | └─ defaults/main.yml
| | └─ tasks/main.yml
| | └─ templates/services.yml.j2
| └─ pihole-dns/             # Pi-hole DNS records (API-based)
| | └─ defaults/main.yml
| | └─ tasks/main.yml
| └─ pbs-backup/             # PBS backup jobs (template-based)
| | └─ defaults/main.yml
| | └─ tasks/main.yml
| | └─ templates/jobs.cfg.j2
| └─ proxmox-vm/             # CoreOS VM creation (qm-based)
| | └─ defaults/main.yml
| | └─ tasks/main.yml
| └─ proxmox-lxc/            # LXC container creation (pct-based)
| | └─ defaults/main.yml
| | └─ tasks/main.yml
└─ vars/                     # VM/CT rebuild recipes (passed via -e @)
    └─ vm-110-prod-db.yml
    └─ vm-120-prod-app.yml
    └─ vm-210-dev-db.yml
    └─ vm-220-dev-app.yml
    └─ ct-311-pihole.yml
    └─ ct-312-cloudflared.yml
    └─ ct-313-traefik.yml
└─ apps/                     # Application variable files (deploy-app)
└─ services/                 # Docker Compose service definitions (future)
└─ butane/                   # Butane template files (future)

```

6.2 ansible.cfg

```
ini
```

```
[defaults]
inventory = inventory/hosts.yml
private_key_file = ~/.ssh/id_ed25519
vault_password_file = ~/.vault_pass
host_key_checking = False
retry_files_enabled = False
stdout_callback = default
result_format = yaml
timeout = 30
forks = 10
roles_path = roles

[privilege_escalation]
become = False
```

Key settings explained:

- `vault_password_file` — Points to `~/.vault_pass` (outside project directory, never committed to Git)
- `host_key_checking = False` — Required for Ansible to connect without interactive confirmation
- `forks = 10` — Number of parallel host connections. Initially set to 3 during IPS troubleshooting, restored to 10 after IPS exclusions were added
- `roles_path = roles` — Tells Ansible where to find roles relative to the project root. Added during Phase 5B-2 when roles were not being discovered.
- `stdout_callback = default` with `result_format = yaml` — Clean output format

6.3 inventory/hosts.yml

```
yaml
```

all:

children:

proxmox:

hosts:

pve-1:

ansible_host: 10.10.100.3

ansible_user: root

proxmox_role: prod

pve-2:

ansible_host: 10.10.100.2

ansible_user: root

proxmox_role: dev

pve-3:

ansible_host: 10.10.100.5

ansible_user: root

proxmox_role: infra

coreos_vms:

children:

db_vms:

hosts:

colossus-prod-db1:

ansible_host: 10.10.100.110

proxmox_node: pve-1

vmid: 110

colossus-dev-db1:

ansible_host: 10.10.100.200

proxmox_node: pve-2

vmid: 210

app_vms:

hosts:

colossus-prod-app1:

ansible_host: 10.10.100.120

proxmox_node: pve-1

vmid: 120

colossus-dev-app1:

ansible_host: 10.10.100.220

proxmox_node: pve-2

vmid: 220

infrastructure:

hosts:

pihole:

```
ansible_host: 10.10.100.53
ctid: 311
cloudflared:
ansible_host: 10.10.100.54
ctid: 312
traefik:
ansible_host: 10.10.100.55
ctid: 313
```

```
backup:
hosts:
pbs:
ansible_host: 10.10.100.242
ansible_user: root
```

```
storage:
hosts:
truenas:
ansible_host: 10.10.0.38
ansible_user: admin
```

Note: TrueNAS (10.10.0.38) has SSH disabled. It is listed in inventory for completeness but excluded from playbooks with `hosts: all:!truenas`. Enable SSH via TrueNAS web UI → System → Services when ready to manage via Ansible.

6.4 Group Variables

inventory/group_vars/all.yml (appended to pre-existing file):

```
yml

# — Network & Infrastructure —————
domain: cogmai.com
traefik_ip: 10.10.100.55
pihole_ip: 10.10.100.53
truenas_ip: 10.10.0.38
pbs_ip: 10.10.100.242

# — Default SSH user (overridden per group) —————
ansible_user: root
```

inventory/group_vars/coreos_vms.yml:

```
yml
```

```
---
```

```
ansible_user: core
```

```
ansible_python_interpreter: /usr/bin/python3
```

inventory/group_vars/infrastructure.yml:

```
yaml
```

```
---
```

```
ansible_user: root
```

inventory/group_vars/proxmox.yml (added Phase 5B-2):

```
yaml
```

```
---  
# Proxmox cluster — shared configuration  
# PBS backup jobs (/etc/pve/jobs.cfg is cluster-wide)
```

pbs_backup_jobs:

```
# — PROD —  
- name: backup-prod-db  
  vmid: 110  
  notes: "PROD DB scheduled backup"  
- name: backup-prod-app  
  vmid: 120  
  notes: "PROD App scheduled backup"  
  
# — DEV —  
- name: backup-dev-db-frozen  
  vmid: 200  
  notes: "DEV DB frozen ref scheduled backup"  
- name: backup-dev-db  
  vmid: 210  
  notes: "DEV DB scheduled backup"  
- name: backup-dev-app  
  vmid: 220  
  notes: "DEV App scheduled backup"  
  
# — Infrastructure —  
- name: backup-pihole  
  vmid: 311  
  notes: "Pi-hole scheduled backup"  
- name: backup-cloudflared  
  vmid: 312  
  notes: "Cloudflared scheduled backup"  
- name: backup-traefik  
  vmid: 313  
  notes: "Traefik scheduled backup"
```

Critical lesson: Group variables MUST be in `inventory/group_vars/` (inside the inventory directory), not in a top-level `group_vars/` directory. The `ansible` CLI ad-hoc commands pick up `group_vars` from the current working directory, but `ansible-playbook` looks relative to the playbook file location. Placing them inside the inventory directory ensures they're always found regardless of where commands are run from.

6.5 Host Variables (added Phase 5B-2)

inventory/host_vars/traefik.yml — Traefik route definitions:

yaml

traefik_routes:

— PROD (external + internal) —————

- name: colossus-legal-frontend
host: "colossus-legal.{{ domain }}"
backend_url: "http://10.10.100.120:5473"
external: **true**
- name: colossus-legal-api
host: "colossus-legal-api.{{ domain }}"
backend_url: "http://10.10.100.120:3001"
external: **true**

— DEV (internal only) —————

- name: colossus-legal-dev
host: "colossus-legal-dev.{{ domain }}"
backend_url: "http://10.10.100.220:5473"
- name: colossus-legal-api-dev
host: "colossus-legal-api-dev.{{ domain }}"
backend_url: "http://10.10.100.220:3001"

— Infrastructure —————

- name: traefik-dashboard
host: "traefik.{{ domain }}"
service: api@internal

inventory/host_vars/pihole.yml — Pi-hole DNS records:

yaml

pihole_dns_records:

— Proxmox nodes —————

- "10.10.100.3 pve-1.cogmai.com"
- "10.10.100.2 pve-2.cogmai.com"
- "10.10.100.5 pve-3.cogmai.com"

— PROD apps (via Traefik) —————

- "10.10.100.55 colossus-legal.cogmai.com"
- "10.10.100.55 colossus-legal-api.cogmai.com"

— DEV apps —————

- "10.10.100.220 colossus-legal-dev.cogmai.com"
- "10.10.100.55 colossus-legal-api-dev.cogmai.com"

— Databases —————

- "10.10.100.110 colossus-prod-db1.cogmai.com"
- "10.10.100.200 colossus-dev-db1.cogmai.com"

— Infrastructure —————

- "10.10.100.53 pihole.cogmai.com"
- "10.10.100.55 traefik.cogmai.com"

Known drift: `colossus-legal-dev.cogmai.com` points to 10.10.100.220 (direct to VM) instead of 10.10.100.55 (Traefik). This was codified as-is during Phase 5B-2 per the principle of documenting current state first, correcting later.

6.6 Ansible Vault

Encrypted secrets are stored in `secrets/vault.yml`:

bash


```
# Create (prompts for vault password and opens editor):
ansible-vault create secrets/vault.yml

# Contents (fill in real values):
# ---
# vault_neo4j_password: "YOUR_NEO4J_PASSWORD"
# vault_cloudflare_dns_api_token: "YOUR_CF_TOKEN"
# vault_cloudflare_tunnel_token: "YOUR_TUNNEL_TOKEN"
# vault_ghcr_username: "YOUR_GITHUB_USERNAME"
# vault_ghcr_token: "YOUR_GHCR_TOKEN"
# vault_anthropic_api_key: "" # Future

# Auto-decrypt via password file:
echo 'YOUR_VAULT_PASSWORD' > ~/.vault_pass
chmod 600 ~/.vault_pass
```

The `vault_password_file` setting in `ansible.cfg` points to `~/.vault_pass`, allowing all vault operations without interactive password entry.

Verify: `ansible-vault view secrets/vault.yml` should display contents without prompting.

6.7 Validation Playbook

playbooks/gather-facts.yml:

```
yaml
---
- name: Gather facts from all Colossus hosts
  hosts: all:!truenas
  gather_facts: true
  tasks:
    - name: Display host summary
      ansible.builtin.debug:
        msg: "{{ inventory_hostname }}" | "{{ ansible_distribution | default('N/A') }}" | "{{ ansible_distribution_version | default('') }}"
```

Expected output (all 11 hosts):

Host	Distribution	Version
pve-1	Debian	13.3
pve-2	Debian	13.3
pve-3	Debian	13.3
colossus-prod-db1	Fedora	43
colossus-dev-db1	Fedora	43
colossus-prod-app1	Fedora	43
colossus-dev-app1	Fedora	43
pihole	Debian	12.x
cloudflared	Debian	12.x
traefik	Debian	12.x
pbs	Proxmox BS	-

7. Ansible Connectivity Verification

7.1 Ad-hoc Ping (Quick Test)

```
bash
ansible all --limit 'truenas' -m ping
```

All 10 managed hosts should return `pong`.

7.2 Full Facts Playbook (Comprehensive Test)

```
bash

# Serial (one host at a time — safest for troubleshooting):
ansible-playbook playbooks/gather-facts.yml --forks 1

# Parallel (full speed):
ansible-playbook playbooks/gather-facts.yml
```

7.3 Idempotency Check

Run the playbook twice. Second run should show `changed=0` for every host:

```
bash

ansible-playbook playbooks/gather-facts.yml
# Expected: ok=2, changed=0 for all 11 hosts
```

8. UniFi UDM SE — IDS/IPS Detection Exclusions

8.1 The Problem

The UniFi UDM SE had Intrusion Prevention (IPS) enabled with "Notify and Block" mode. The IPS was treating legitimate cross-VLAN traffic between the homelab VLAN (10.10.100.0/24) and the main/NAS network (10.10.0.0/24) as intrusion attempts, intermittently dropping packets.

Symptoms:

- Random SSH timeouts when running `ansible-playbook` — different hosts failed each run
- Even `--forks 1` (serial) failed randomly
- `ping` and `nc -zv` to affected hosts always succeeded
- SSH would hang during key exchange or after authentication

8.2 Root Cause Evidence

System Log → Security Detection in the UniFi controller showed hundreds of blocks:

Source	Destination	Meaning
10.10.100.2 → 10.10.0.38	pve-2 → TrueNAS	NFS/PBS backup sync
10.10.100.3 → 10.10.0.38	pve-1 → TrueNAS	NFS/PBS backup sync
10.10.100.5 → 10.10.0.38	pve-3 → TrueNAS	NFS/PBS backup sync
10.10.0.99 → 10.10.100.53	(unknown) → pihole	DNS queries
10.10.0.99 → 10.10.100.120	(unknown) → prod-app1	Application traffic

The IPS was blocking Proxmox ↔ TrueNAS backup traffic approximately every 15 minutes. This was causing silent backup disruptions that were not previously noticed, in addition to the Ansible SSH timeouts.

8.3 IPS Configuration (Before Fix)

- **Location:** UniFi Controller → CyberSecure → Protection
- **Intrusion Prevention:** ON
- **Detection Mode:** Notify and Block
- **Selected Networks:** Default
- **Active Detections:** "Attacks and Reconnaissance" (3 of 7 enabled), "Hacking and Exploits" (3 of 5 enabled)

8.4 The Fix — Detection Exclusions

Location: UniFi Controller → CyberSecure → Protection → Detection Exclusions → Create New

Added two exclusions:

1. **10.10.100.0/24** — Homelab VLAN (all Proxmox nodes, VMs, CTs)
2. **10.10.0.0/24** — Main/NAS network (TrueNAS, workstation)

These tell the IPS to not inspect traffic originating from or destined to these subnets. IPS continues to protect against external (internet-facing) threats.

8.5 Verification

After adding exclusions, the Ansible playbook immediately succeeded for all 11 hosts:

```
bash

# Serial — 11/11 success:
ansible-playbook playbooks/gather-facts.yml --forks 1

# Parallel (forks=10) — 11/11 success, changed=0 (idempotent):
ansible-playbook playbooks/gather-facts.yml
```

8.6 Impact Beyond Ansible

The IPS exclusions also fixed:

- **PBS sync job stability** — Proxmox → TrueNAS NFS traffic was being intermittently blocked every 15 minutes
- **Cross-VLAN service reliability** — Any traffic between the homelab VLAN and main network is no longer subject to false-positive IPS blocks
- **DNS resolution** — Traffic to pihole (10.10.100.53) from the main network was occasionally blocked

8.7 General Guidance: Homelab IPS

IPS is designed to protect against external threats. Internal homelab traffic between your own subnets should be excluded from IPS inspection for two reasons:

1. **False positives** — Automated tools (Ansible, cron jobs, backup sync) generate traffic patterns that look like scans/attacks to signature-based IPS
2. **Performance** — Inspecting every internal packet adds latency and CPU overhead with no security benefit

Keep IPS enabled for internet-facing traffic. Exclude all internal management and service subnets.

9. Phase 5B-2 — Infrastructure Roles

9.1 Role Architecture

Phase 5B-2 codified existing manually-configured infrastructure into reusable Ansible roles. The roles fall into two architectural categories:

Configuration roles (run repeatedly to enforce desired state):

Role	Target	Approach	Idempotency Model
<code>traefik-route</code>	CT-313	Template <code>services.yml</code> , Traefik hot-reloads	File comparison — <code>changed=0</code> when template output matches
<code>pihole-dns</code>	CT-311	Pi-hole v6 REST API (GET/PUT/DELETE)	Set comparison — desired vs current records
<code>pbs-backup</code>	Proxmox cluster	Template <code>/etc/pve/jobs.cfg</code>	File comparison — <code>changed=0</code> when template output matches

Provisioning roles (run once to create resources):

Role	Target	Approach	Idempotency Model
<code>proxmox-vm</code>	Any Proxmox node	<code>qm</code> commands with pre-flight checks	Skip-if-exists — <code>qm status \$VMID</code> check
<code>proxmox-lxc</code>	Any Proxmox node	<code>pct</code> commands with pre-flight checks	Skip-if-exists — <code>pct status \$CTID</code> check

Deployment roles (pre-existing from Phase 5B design):

Role	Target	Approach	Idempotency Model
deploy-app	CoreOS app VMs	Pull images, reload systemd, health checks	Manifest-tracked versioning
colossus-legal	CoreOS app VMs	Quadlet files, env files, frontend config	Template comparison

9.2 traefik-route Role

What it manages: The Traefik dynamic routing configuration at `/etc/traefik/dynamic/services.yml` on CT-313.

How it works: Ansible owns the entire `services.yml` file. All routes are defined as variables in `inventory/host_vars/traefik.yml`. A Jinja2 template renders the complete file. Traefik watches the file for changes and hot-reloads with zero downtime — no service restart needed.

Current routes (5):

Route	Host	Backend	External
colossus-legal-frontend	colossus-legal.cogmai.com	http://10.10.100.120:5473	Yes
colossus-legal-api	colossus-legal-api.cogmai.com	http://10.10.100.120:3001	Yes
colossus-legal-dev	colossus-legal-dev.cogmai.com	http://10.10.100.220:5473	No
colossus-legal-api-dev	colossus-legal-api-dev.cogmai.com	http://10.10.100.220:3001	No
traefik-dashboard	traefik.cogmai.com	api@internal	No

Template details: Routes marked `external: true` get both an HTTPS router (TLS with Let's Encrypt wildcard cert via DNS-01/Cloudflare) and an HTTP router (priority 10) for Cloudflare Tunnel traffic. Internal-only routes get HTTPS only. A catch-all HTTP router (priority 1) redirects all other HTTP traffic to HTTPS.

Usage:

```
bash
```

```
# Dry-run (shows diff):
ansible-playbook playbooks/manage-traefik.yml --check --diff

# Execute:
ansible-playbook playbooks/manage-traefik.yml

# Add a new route: edit inventory/host_vars/traefik.yml, then re-run
```

Validation results: First run (`changed=1`) (cosmetic header added), second run (`changed=0`) (idempotent). Traefik process verified running and listening on port 443.

9.3 pi-hole-dns Role

What it manages: DNS records in Pi-hole v6 via its REST API on CT-311.

How it works: The role fetches current records from the Pi-hole API (`GET /api/config/dns/hosts`), computes the diff between desired and actual state, then uses (`PUT`) to add missing records and (`DELETE`) to remove extras. This is a declarative, API-driven approach — no file templating.

Pi-hole v6 API details:

- Config location: `/etc/pihole/pihole.toml` → `dns.hosts` array (authoritative)
- Legacy `custom.list` still exists but is stale — the API is the source of truth
- API base: `http://localhost/api/config/dns/hosts`
- No authentication required (sessions auto-valid on localhost)
- Record format: HOSTS-style strings, e.g., `"10.10.100.55 traefik.cogmai.com"`
- Spaces in PUT/DELETE paths must be URL-encoded as `%20`

Current records (11): See Section 6.5 (`inventory/host_vars/pihole.yml`) for the full list.

Usage:

```
bash

# Dry-run (shows planned additions/removals):
ansible-playbook playbooks/manage-pihole.yml --check

# Execute:
ansible-playbook playbooks/manage-pihole.yml

# Add a DNS record: edit inventory/host_vars/pihole.yml, then re-run
```

Validation results: First run `changed=0` (records already matched). `dig` validation task confirmed DNS resolution working.

9.4 pbs-backup Role

What it manages: Proxmox backup job definitions in `/etc/pve/jobs.cfg`.

How it works: Same template-based approach as `traefik-route`. All 8 backup jobs are defined in `inventory/group_vars/proxmox.yml` and rendered into `jobs.cfg` via a Jinja2 template. The template preserves the exact tab-indented format that Proxmox expects.

Key detail: `/etc/pve/jobs.cfg` lives on the Proxmox cluster filesystem (`pmxcfs`), which is shared across all nodes. The playbook uses `run_once: true` to write the file from only one node (whichever Ansible connects to first).

Current backup jobs (8):

Job Name	VMID	Target
backup-prod-db	110	PROD DB (pve-1)
backup-prod-app	120	PROD App (pve-1)
backup-dev-db-frozen	200	DEV DB frozen ref
backup-dev-db	210	DEV DB (pve-2)
backup-dev-app	220	DEV App (pve-2)
backup-pihole	311	Pi-hole (pve-3)
backup-cloudflared	312	cloudflared (pve-3)
backup-traefik	313	Traefik (pve-3)

All jobs use: `schedule: daily`, `compress: zstd`, `mode: snapshot`, `storage: pbs-1`.

Usage:

```
bash
```



```
# Dry-run:
ansible-playbook playbooks/manage-pbs-backups.yml --check --diff

# Execute:
ansible-playbook playbooks/manage-pbs-backups.yml

# Add a backup job: add entry to inventory/group_vars/proxmox.yml, then re-run
```

Validation results: First run (changed=1) (removed one cosmetic extra blank line from original file), second run (changed=0) (idempotent). Validation task confirmed 8 (vzdump:) entries in the file.

9.5 proxmox-vm Role

What it manages: Creation of Fedora CoreOS VMs on Proxmox hosts via (qm) commands.

How it works: The role checks if the VMID already exists ((qm status)). If it does, the entire creation block is skipped. If not, it runs pre-flight checks (CoreOS image, Ignition file, directory mappings), then executes the same sequence of (qm) commands documented in the CoreOS VM Creation Runbook: create VM (q35 machine type), import QCOW2 disk, grow disk, attach cloud-init drive, set boot order, enable serial console, configure Ignition via cicustom, and optionally attach virtiofs shares.

Parameters are passed via vars files in the (vars/) directory, not in inventory. This is because VM definitions aren't tied to a specific inventory host — they're parameters for a one-shot provisioning workflow.

Current vars files:

File	VMID	Name	Node	virtiofs
(vars/vm-110-prod-db.yml)	110	colossus-prod-db1	pve-1	3 shares (prod-db-*)
(vars/vm-120-prod-app.yml)	120	colossus-prod-app1	pve-1	None
(vars/vm-210-dev-db.yml)	210	colossus-dev-db1	pve-2	3 shares (db-*)
(vars/vm-220-dev-app.yml)	220	colossus-dev-app1	pve-2	None

Usage:

```
bash
```

```
# Create PROD DB VM (after destroy, or for fresh build):
ansible-playbook playbooks/create-vm.yml -l pve-1 -e @vars/vm-110-prod-db.yml

# Create a future AI app VM:
ansible-playbook playbooks/create-vm.yml -l pve-2 -e @vars/vm-230-dev-ai-app.yml

# Safe to run against existing VMs (skips with message):
ansible-playbook playbooks/create-vm.yml -l pve-1 -e @vars/vm-110-prod-db.yml
# → "VM 110 (colossus-prod-db1) already exists. No changes made."
```

Prerequisites for VM creation:

- 1. ZFS pool and datasets created on target node
- 2. Proxmox directory mappings created (for virtiofs VMs)
- 3. Butane config transpiled to Ignition and copied to `/var/coreos/snippets/`
- 4. CoreOS QCOW2 image downloaded to `/var/coreos/images/`

These prerequisites are checked by the role's pre-flight tasks and will fail with descriptive error messages if not met.

9.6 proxmox-lxc Role

What it manages: Creation of LXC containers on Proxmox hosts via `pct` commands.

How it works: Same skip-if-exists pattern as `proxmox-vm`. Checks `pct status`, skips if the CTID exists. Otherwise: finds the latest Debian 12 template, creates the container with static IP, starts it, and verifies it's running.

Current vars files:

File	CTID	Hostname	IP	Memory
<code>vars/ct-311-pihole.yml</code>	311	pihole	10.10.100.53/24	512MB
<code>vars/ct-312-cloudflared.yml</code>	312	cloudflared	10.10.100.54/24	256MB
<code>vars/ct-313-traefik.yml</code>	313	traefik	10.10.100.55/24	256MB

Usage:

```
bash
```

```
# Create Pi-hole container (after destroy, or for fresh build):
ansible-playbook playbooks/create-lxc.yml -l pve-3 -e @vars/ct-311-pihole.yml

# Safe to run against existing containers:
ansible-playbook playbooks/create-lxc.yml -l pve-3 -e @vars/ct-311-pihole.yml
# → "CT 311 (pihole) already exists. No changes made."
```

Note: The LXC role creates the bare container only. Application-specific setup (installing Pi-hole, cloudflared, Traefik) is handled by separate installation scripts, not by this role. Future work may codify those installation steps as additional Ansible roles.

9.7 deploy-app + colossus-legal Roles (Pre-existing)

These roles were created during the initial Phase 5B design (before 5B-2) and implement the application deployment workflow:

- **deploy-app** — Generic orchestrator: reads current manifest, logs in to GHCR, pulls images, delegates to app-specific role, reloads systemd, restarts containers, runs health checks, writes deployment manifest. Supports rollback via manifest's `previous_version` field.
- **colossus-legal** — App-specific role: writes Quadlet `.container` files, environment files, and frontend `config.js`. Plugged into `deploy-app` via the `app_def.role` variable.

Usage:

```
bash

# Deploy to DEV:
ansible-playbook playbooks/deploy-app.yml -e app=colossus-legal -e version=v1.0.0 -l dev

# Deploy to PROD (with confirmation prompt):
ansible-playbook playbooks/deploy-app.yml -e app=colossus-legal -e version=v1.0.0 -l prod
```

This separation means adding a new application (e.g., `colossus-ai`) requires only: create a `colossus-ai` role with its templates, add an app definition to `group_vars/all.yml`, and run the same `deploy-app.yml` playbook.

10. Complete Host Connectivity Reference

10.1 All Managed Hosts

Host	Inventory Name	IP	SSH User	Group	Notes
pve-1	pve-1	10.10.100.3	root	proxmox	PROD node
pve-2	pve-2	10.10.100.2	root	proxmox	DEV node
pve-3	pve-3	10.10.100.5	root	proxmox	Infra node
VM-110	colossus-prod-db1	10.10.100.110	core	coreos_vms.db_vms	PROD databases
VM-210	colossus-dev-db1	10.10.100.200	core	coreos_vms.db_vms	DEV databases
VM-120	colossus-prod-app1	10.10.100.120	core	coreos_vms.app_vms	PROD applications
VM-220	colossus-dev-app1	10.10.100.220	core	coreos_vms.app_vms	DEV applications
CT-311	pihole	10.10.100.53	root	infrastructure	DNS
CT-312	cloudflared	10.10.100.54	root	infrastructure	Cloudflare Tunnel
CT-313	traefik	10.10.100.55	root	infrastructure	Reverse proxy
VM-900	pbs	10.10.100.242	root	backup	Proxmox Backup Server

10.2 Unmanaged Hosts

Host	IP	Reason
TrueNAS	10.10.0.38	SSH disabled; manage via web UI or API

11. Playbook Quick Reference

11.1 Configuration Playbooks (Run Regularly)

Playbook	What It Does	Dry-Run
<code>manage-traefik.yml</code>	Deploy Traefik routes	<code>--check --diff</code>

Playbook	What It Does	Dry-Run
<code>manage-pihole.yml</code>	Sync Pi-hole DNS records	<code>--check</code>
<code>manage-pbs-backups.yml</code>	Deploy PBS backup job config	<code>--check --diff</code>
<code>gather-facts.yml</code>	Validate connectivity to all hosts	N/A

11.2 Deployment Playbooks (On Demand)

Playbook	What It Does	Key Vars
<code>deploy-app.yml</code>	Deploy an application version	<code>-e app=<name> -e version=<tag> -l <env></code>

11.3 Provisioning Playbooks (Run Once)

Playbook	What It Does	Key Vars
<code>create-vm.yml</code>	Create a CoreOS VM	<code>-l <node> -e @vars/vm-<id>.yml</code>
<code>create-lxc.yml</code>	Create an LXC container	<code>-l <node> -e @vars/ct-<id>.yml</code>

11.4 Adding New Infrastructure

New application route (e.g., colossus-ai):

1. Add route entry to `inventory/host_vars/traefik.yml`
2. Add DNS record to `inventory/host_vars/pihole.yml`
3. Run `ansible-playbook playbooks/manage-traefik.yml`
4. Run `ansible-playbook playbooks/manage-pihole.yml`

New VM:

1. Create `vars/vm-<id>-<name>.yml` with VM parameters
2. Prepare prerequisites (ZFS, directory mappings, Ignition file)
3. Run `ansible-playbook playbooks/create-vm.yml -l <node> -e @vars/vm-<id>-<name>.yml`
4. Add backup job to `inventory/group_vars/proxmox.yml`
5. Run `ansible-playbook playbooks/manage-pbs-backups.yml`

New LXC container:

1. Create `vars/ct-<id>-<name>.yaml` with container parameters
 2. Run `ansible-playbook playbooks/create-lxc.yml -l <node> -e @vars/ct-<id>-<name>.yaml`
 3. Install application-specific software via separate script/role
 4. Add backup job to `inventory/group_vars/proxmox.yml`
 5. Run `ansible-playbook playbooks/manage-pbs-backups.yml`
-

12. Troubleshooting Reference

12.1 SSH Timeouts to Random Hosts

Check first: UniFi IDS/IPS — System Log → Security Detection for blocks involving homelab IPs.

Then check:

```
bash

# Is the host reachable at all?
ping -c 2 <host_ip>

# Is SSH port open?
nc -zv -w3 <host_ip> 22

# Verbose SSH (look for where it stalls):
ssh -v -o ConnectTimeout=5 <user>@<host_ip> "hostname"

# Kill stale multiplexing socket:
rm ~/.ssh/cm-<user>@<host_ip>:22
```

12.2 CoreOS VM — "No Python" Error

```
colossus-prod-db1 | FAILED! => {"module_stderr": "/bin/sh: python3: command not found"}
```

Fix:

```
bash

ssh core@<host_ip> "sudo rpm-ostree install python3 libselenium-python3 --apply-live"
```

12.3 LXC Container — SSH Connection Refused

```
pihole | UNREACHABLE! => {"msg": "Connection refused"}
```

Fix (from pve-3):

```
bash  
  
ssh root@10.10.100.5 "pct exec <ctid> -- systemctl enable --now ssh"
```

12.4 Ansible Playbook Uses Wrong User

```
UNREACHABLE! => Permission denied (publickey,password)
```

Check: Are group_vars in `inventory/group_vars/`, not top-level `group_vars/`? Verify with:

```
bash  
  
ansible-inventory --host <hostname> | grep ansible_user
```

12.5 Vault Decryption Fails

```
ERROR! vault_password_file ... already exists
```

Fix: Ensure `vault_password_file` is under the `[defaults]` section in `ansible.cfg`, not under `[privilege_escalation]`.

12.6 Role Not Found

```
ERROR! the role 'traefik-route' was not found
```

Fix: Ensure `roles_path = roles` is present in the `[defaults]` section of `ansible.cfg`. This was added during Phase 5B-2 when Ansible could not discover roles in the `roles/` directory.

12.7 Pi-hole API Returns Empty Response

If the `pihole-dns` role's DELETE task fails with an unexpected response, the Pi-hole API returns empty responses (no JSON) for successful deletes. The role handles this by accepting both status 200 and 204.

12.8 Proxmox jobs.cfg Format Issues

The `/etc/pve/jobs.cfg` file uses **literal tab characters** for indentation, not spaces. If the file is hand-edited or the template is modified, ensure tabs are preserved. The `pbs-backup` role template uses heredoc syntax to maintain real tab characters.

13. Success Criteria — Final Status

13.1 Phase 5B-1 (Foundation)

#	Criterion	Status
1	Ansible inventory covers all 9 VMs/CTs + PBS	✓ 11 hosts (10 managed + TrueNAS listed)
2	<code>ansible all -m ping</code> succeeds for every managed host	✓ 10/10
3	Ansible Vault stores all secrets	✓ Neo4j, Cloudflare tokens, GHCR credentials
4	Facts playbook runs successfully against all hosts	✓ 11/11 (serial + parallel)
5	Idempotent execution (changed=0 on second run)	✓ Confirmed

13.2 Phase 5B-2 (Infrastructure Roles)

#	Criterion	Status
1	traefik-route: template-based, idempotent, hot-reload	✓ changed=0 on second run
2	pihole-dns: API-based, declarative, idempotent	✓ changed=0, dig validation passed
3	pbs-backup: template-based, cluster-wide, idempotent	✓ changed=0 on second run
4	proxmox-vm: qm-based, skip-if-exists	✓ Validated against existing VM-110
5	proxmox-lxc: pct-based, skip-if-exists	✓ Validated against existing CT-311
6	deploy-app: confirmed pre-existing, operational	✓ Existing pattern validated
7	All vars files document current infrastructure	✓ 4 VMs + 3 CTs parameterized

14. Key Lessons Learned

14.1 Phase 5B-1 Lessons

1. **CoreOS + Ansible:** The canonical approach is `rpm-ostree install python3 libselenium-python3 --apply-live`. No reboot required. Survives auto-updates.

2. **UniFi IPS + Homelab:** Internal subnets MUST be excluded from IDS/IPS detection. The IPS sees automated tools (Ansible, cron backups, DNS queries) as attacks and silently drops packets. This can cause intermittent, hard-to-diagnose failures.
3. **SSH multiplexing:** Essential for Ansible stability when managing many hosts. Configure at the subnet level (`Host 10.10.100.*`) with `ControlPersist` for session reuse.
4. **group_vars location:** Must be inside `inventory/group_vars/` for playbooks to find them reliably. Top-level `group_vars/` works for ad-hoc commands but not for playbooks in subdirectories.
5. **Debian SSH service name:** `ssh`, not `sshd`. The `sshd.service` unit exists but is an alias — `systemctl enable` refuses to operate on aliases.
6. **ssh-copy-id reliability:** Don't trust "keys already exist" messages. Always verify with an actual key-based SSH connection. Manual key deployment via `echo` + `cat` is more reliable through multi-hop scenarios.
7. **LXC containers need SSH:** Ansible requires direct SSH access. `pct exec` delegation through Proxmox hosts is not sufficient. Install `openssh-server` and deploy keys on all LXCs that Ansible will manage.

14.2 Phase 5B-2 Lessons

8. **Configuration vs Provisioning roles:** Configuration roles (`traefik-route`, `pihole-dns`, `pbs-backup`) enforce desired state on every run with `changed=0` idempotency. Provisioning roles (`proxmox-vm`, `proxmox-lxc`) create resources once and skip on subsequent runs. The architectural pattern differs: templates and API calls vs imperative commands wrapped in existence checks.
9. **Template-based vs API-based roles:** Template-based roles (`traefik-route`, `pbs-backup`) are simpler — Ansible's built-in `template` module handles diff, backup, and idempotency. API-based roles (`pihole-dns`) require explicit state comparison logic but are necessary when the service doesn't expose its config as files.
10. **Jinja2 whitespace in Ansible:** Ansible enables `trim_blocks` and `lstrip_blocks` by default on Jinja2 templates. This means `{% for %}` and `{% if %}` tags don't produce blank lines. Test templates with matching settings to preview actual output.
11. **Tab characters in templates:** Proxmox's `jobs.cfg` uses literal tab indentation. Jinja2 templates must preserve these — use heredoc syntax or verify with `cat -A` to confirm `^I` (tab) rather than spaces.
12. **Pi-hole v6 API:** The REST API at `/api/config/dns/hosts` is the authoritative interface. The legacy `custom.list` file may be stale. PUT requires URL-encoding spaces as `%20`. DELETE returns empty responses (no JSON body), so accept status 200 or 204.
13. **vars/ directory for provisioning:** VM and LXC definitions go in `vars/` (loaded via `-e @vars/file.yml`) rather than `host_vars/` or `group_vars/`, because they're parameters to a provisioning workflow, not properties of an existing inventory host. These files also serve as disaster recovery documentation.

14. **roles_path in ansible.cfg:** Must be set explicitly (`roles_path = roles`) for Ansible to discover roles in the project's `roles/` directory. Without this, playbooks fail with "role not found" errors.
 15. **Cluster filesystem awareness:** `/etc/pve/jobs.cfg` is shared across all Proxmox nodes via pmxcfs. The `pbs-backup` playbook uses `run_once: true` to avoid writing the same file from multiple nodes simultaneously.
-

15. References

Document	Purpose
<code>COLOSSUS_PHASE5B_ANSIBLE_DESIGN_v1.md</code>	Full Ansible design and implementation plan
<code>COLOSSUS_HOMELAB_MASTER_CONTEXT_v5.md</code>	Current infrastructure state
<code>PHASE5B1_SESSION_TRANSITION.md</code>	Phase 5B-1 session transition record
<code>COLOSSUS_COREOS_VM_CREATION_RUNBOOK_v1.md</code>	CoreOS VM provisioning (Butane/Ignition)
<code>COLOSSUS_TRUENAS_INTEGRATION_DESIGN_v1.md</code>	TrueNAS backup architecture
rpm-ostree apply-live docs	Official apply-live architecture
UniFi IDS/IPS	UniFi Threat Management configuration
Pi-hole v6 API	Pi-hole REST API documentation